Project: The AI Data Detective

Applying Linear Algebra in Python with Numpy

Time limit: 60 Minutes — **Prerequisites:** Basic Python, Google Colab / Replit

Mission Briefing

Today, you are an AI Engineer. You will use the `numpy` library to translate the Linear Algebra concepts from your AIM202 course into real working AI algorithms. You must complete three operations:

1. **Word Embeddings:** Find related words using Cosine Similarity.
2. **Neural Networks:** Compute a forward pass of a linear layer.
3. **Cryptography:** Decrypt a secret message using Matrix Inverses.

Task 1: Semantic Word Matcher (Cosine Similarity)

In AI, words are represented as vectors. To find how similar two words are, we don't look at their exact coordinates, but rather the angle between them using **Cosine Similarity**. The formula is:

$$\text{Similarity} = \frac{\vec{A} \cdot \vec{B}}{||\vec{A}|| \cdot ||\vec{B}||}$$

Your Goal: Complete the function to find the similarity between the word "Karak" and "Gahwa", versus "Karak" and "Laptop".

```python
import numpy as np

# Word Vectors (Embeddings)
karak = np.array([0.9, 0.8, 0.2])
gahwa = np.array([0.8, 0.9, 0.1])
laptop = np.array([0.1, 0.0, 0.9])

def calculate_similarity(vec_a, vec_b):
    # Step 1: Calculate the dot product of vec_a and vec_b
    dot_prod = np._________

    # Step 2: Calculate the magnitudes (L2 Norm)
    mag_a = np.linalg.norm(vec_a)
    mag_b = np.linalg.norm(________)

    # Step 3: Return the cosine similarity
    return dot_prod / (________ * ________)

print("Karak vs Gahwa:", calculate_similarity(karak, gahwa))
print("Karak vs Laptop:", calculate_similarity(karak, laptop))
```

Task 2: Neural Network Linear Layer

In a neural network, a linear layer processes inputs by multiplying an input vector ($\vec{x}$) by a Weight matrix (W). Your neural network has 3 inputs and 2 outputs.

Your Goal: Perform the matrix multiplication $Y = W \cdot \vec{x}$.

```
# Neural Network Weights (2x3 Matrix)
W = np.array([[1, -1, 2],
              [0,  3, 1]])

# Input features (3x1 Vector)
x = np.array([2,
              1,
              4])

# Step 1: Use np.dot() to multiply the Weight matrix by the input vector
output_y = np._________(W, x)

print("Neural Network Output:", output_y)
```

Task 3: Matrix Decryption (Inverse)

You intercepted an encrypted vector C. You know the encryption key was Matrix K. To solve the linear system $K \cdot M = C$ and find the original message M, you must multiply by the inverse: $M = K^{-1} \cdot C$.

Your Goal: Find the inverse of K and decode C.

```
# Encryption Key Matrix K
K = np.array([[4, 3],
              [3, 2]])

# Encrypted Message Vector C
C = np.array([[10],
              [7]])

# Step 1: Find the inverse of Matrix K
# Hint: Use np.linalg.inv()
K_inv = np._________________(K)

# Step 2: Multiply K_inv by C to get the Message M
message_M = np.dot(________, ________)

print("Decoded Message:\n", np.round(message_M))
```

When you have completed all three tasks and successfully decoded the message, show your terminal output to your instructor!

For each task, provide an explanation of what does that answer means and interpret and expound on the output the program. Submit a pdf file within two(2) days